Module 3 - Log Analysis and Incident Detection

3.1 - Log Normalization and Parsing

Objective

To understand how to collect, normalize, and parse logs from various sources using Python. This module focuses on building a strong foundation for log analysis as part of defensive security operations, including the use of Regular Expressions (Regex), log parsing libraries, and structured output handling for downstream analysis.

---

1. Introduction to Logs in Security

Logs are essential in security operations, providing evidence and insights into system behavior, user actions, and possible security incidents. However, logs come in **varied formats**, often with **inconsistent structures**, making parsing a critical step before any meaningful analysis.

Common log sources:

- Web servers (Apache, Nginx)
- Firewalls (pfSense, Cisco)
- Operating systems (Windows Event Logs, Linux syslog)
- Security tools (IDS/IPS, AV, EDR)
- Cloud providers (AWS CloudTrail, GCP Audit Logs)

---

2. Understanding Log Formats

Logs can be structured, semi-structured, or unstructured. Let's briefly define these:

- **Structured logs**: JSON, CSV — easy to parse.
- **Semi-structured logs**: Syslog, Apache/Nginx — human-readable but consistent.
- **Unstructured logs**: Custom formats — require regex or custom parsers.

Example: Apache access log

```
127.0.0.1 - - [13/Jun/2025:12:22:01 +0000] "GET /index.html HTTP/1.1" 200 1024
```

Example: Linux syslog

```
Jun 13 12:22:01 server1 sshd[12345]: Failed password for invalid user root from 192.168.1.10 port 2233 ssh2
```

---

3. Basic Log File Reading in Python

```python
# Read a log file line by line
with open("auth.log", "r") as logfile:
    for line in logfile:
        print(line.strip())
```

> This is the foundational step before applying any parsing or transformation logic.

---

4. Using Regular Expressions for Parsing

Python's `re` module is powerful for parsing semi-structured and unstructured log formats.

Example: Parse a Linux SSH failure line

```
import re

log_line = 'Jun 13 12:22:01 server1 sshd[12345]: Failed password for invalid user root from 192.168.1.10 port
2233 ssh2'

# Create a regex pattern with named groups
pattern = re.compile(
    r'(?P<month>\w+)\s+(?P<day>\d+)\s+(?P<time>\d+:\d+:\d+)\s+(?P<host>\S+)\s+sshd\[\d+\]:\s+Failed password for
(invalid user )?(?P<user>\S+)\s+from\s+(?P<ip>\d+\.\d+\.\d+\.\d+)\s+port\s+(?P<port>\d+)\s+ssh2'
)

match = pattern.search(log_line)

if match:
    print(match.groupdict())
```

Output:

```
{
  "month": "Jun",
  "day": "13",
  "time": "12:22:01",
  "host": "server1",
  "user": "root",
  "ip": "192.168.1.10",
  "port": "2233"
}
```

> Named groups help convert raw log lines into structured dictionaries easily, suitable for export to JSON or database.

5. Building a Log Normalizer Class

You can generalize log normalization using Python classes.

```
class SSHLogParser:
    def __init__(self):
        self.pattern = re.compile(
            r'(?P<month>\w+)\s+(?P<day>\d+)\s+(?P<time>\d+:\d+:\d+)\s+(?P<host>\S+)\s+sshd\[\d+\]:\s+Failed
password for (invalid user )?(?P<user>\S+)\s+from\s+(?P<ip>\d+\.\d+\.\d+\.\d+)\s+port\s+(?P<port>\d+)\s+ssh2'
        )

    def parse_line(self, line):
        match = self.pattern.search(line)
        if match:
            return match.groupdict()
        return None
```

Example usage:

```
parser = SSHLogParser()

with open("auth.log", "r") as f:
    for line in f:
        result = parser.parse_line(line)
        if result:
            print(result)
```

6. Parsing Structured Logs (e.g., JSON)

Some logs are already in JSON format. In such cases, parsing is trivial.

```
import json

with open("cloudtrail.log", "r") as log:
    for line in log:
        event = json.loads(line)
        print(event["eventName"], event["userIdentity"]["arn"])
```

> For JSON logs, the challenge becomes filtering and transforming rather than parsing structure.

---

7. Parsing CSV-based Logs

Some tools export logs in CSV format:

```
import csv

with open("firewall.csv", newline="") as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        print(row["source_ip"], "->", row["destination_ip"], row["action"])
```

> Use `DictReader` to map headers to values automatically.

---

8. Parsing Logs Using `pandas`

`pandas` is efficient for tabular logs (CSV, JSON, TSV):

```python
import pandas as pd

df = pd.read_csv("ids_logs.csv")
suspicious = df[df["signature"].str.contains("SQL Injection")]
print(suspicious[["timestamp", "src_ip", "signature"]])
```

> This enables filtering, aggregation, and export in one unified pipeline.

---

9. Log Normalization Schema Design

It's best practice to convert logs to a normalized structure like:

```json
{
  "timestamp": "2025-06-13T12:22:01Z",
  "source": "sshd",
  "host": "server1",
  "user": "root",
  "ip": "192.168.1.10",
  "port": 2233,
  "event": "failed_login"
}
```

You can convert parsed fields to this format programmatically.

---

10. Exporting Parsed Logs

To JSON:

```python
import json

with open("parsed_logs.json", "w") as f:
    json.dump(parsed_data_list, f, indent=2)
```

To CSV:

```python
import csv

with open("parsed_logs.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["timestamp", "user", "ip", "port", "event"])
    writer.writeheader()
    writer.writerows(parsed_data_list)
```

---

11. Handling Errors and Corrupted Lines

Always wrap your parsers with exception handling:

```python
def safe_parse(line, parser):
    try:
        return parser(line)
    except Exception as e:
        print("Parsing error:", e)
        return None
```

---

12. Real-world Use Case: SSH Brute Force Detection

Let’s combine parsing + aggregation:

```
from collections import Counter

ips = []

with open("auth.log") as f:
    for line in f:
        match = parser.parse_line(line)
        if match:
            ips.append(match["ip"])

top_ips = Counter(ips).most_common(5)

for ip, count in top_ips:
    print(f"{ip}: {count} failed attempts")
```

Practice Exercise

Create a Python script that:

1. Reads a syslog file.
2. Parses failed SSH login attempts.
3. Normalizes to a dictionary format.
4. Counts failed attempts by IP.
5. Exports results to a JSON file.

3.2 - Detection of Suspicious Behavior

Objective

To develop a strong understanding of how to detect suspicious behavior in system and network logs using Python. This includes pattern recognition, frequency analysis, anomaly detection, heuristic-based rules, and foundations for behavior-based alerting systems used in SOCs, EDRs, and SIEMs.

---

1. Introduction: What Is Suspicious Behavior in Logs?

In cybersecurity, suspicious behavior refers to actions that deviate from normal user or system activity and may indicate a potential compromise, insider threat, brute-force attack, data exfiltration, or reconnaissance.

Common indicators:

- Repeated failed login attempts
- Access to restricted files
- Privilege escalation
- Lateral movement (e.g., accessing multiple machines)
- Irregular working hours activity
- Large data transfers or downloads
- Use of tools like `netcat`, `curl`, `wget`, `nmap`, etc.

---

2. Detection Strategies

There are multiple approaches to detect suspicious activity:

Approach	Description
Rule-based	Manually defined patterns (e.g., "5 failed logins in 2 minutes")
Threshold-based	Numerical limits (e.g., "over 100MB downloaded")

Approach	Description
Time-based	Activity outside normal working hours
Frequency-based	Unusual spikes in similar events
Heuristic	Known indicators of compromise (IOCs) and attacker TTPs
Behavioral	Detecting deviation from learned normal behavior

We'll focus on **rule-based**, **threshold-based**, and **frequency-based** detection using Python.

---

3. Detecting Brute-force Login Attempts

This example counts repeated failed login attempts by IP.

Sample log line:

```
Jun 14 02:11:41 server sshd[1337]: Failed password for invalid user admin from 192.168.1.50 port 3357 ssh2
```

Code:

```python
import re
from collections import defaultdict

logfile = "auth.log"
failed_attempts = defaultdict(int)

pattern = re.compile(r"Failed password.*from (?P<ip>\d+\.\d+\.\d+\.\d+)")

with open(logfile, "r") as f:
    for line in f:
```

```
        match = pattern.search(line)
        if match:
            ip = match.group("ip")
            failed_attempts[ip] += 1

# Show IPs with 5 or more failures
for ip, count in failed_attempts.items():
    if count >= 5:
        print(f"[ALERT] Suspicious IP: {ip} with {count} failed logins")
```

> This mimics a basic brute-force detection logic found in many intrusion detection systems (IDS).

---

4. Time-based Anomaly Detection

Some attacks happen at odd hours (e.g., 2 AM). Let's flag logins outside business hours (8 AM - 6 PM).

```
import datetime
import re

time_pattern = re.compile(r"(?P<month>\w+)\s+(?P<day>\d+)\s+(?P<time>\d+:\d+:\d+).*Accepted password for (?
P<user>\w+)")
month_to_num = {"Jun": 6}  # Expand as needed

with open("auth.log") as f:
    for line in f:
        match = time_pattern.search(line)
        if match:
            time_str = match.group("time")
            hour = int(time_str.split(":")[0])
```

```
            if hour < 8 or hour > 18:
                print(f"[SUSPICIOUS LOGIN] At {time_str} by {match.group('user')}")
```

---

5. Monitoring Access to Sensitive Files

Let's assume we're watching a file audit log:

```
accessed /etc/shadow by root
accessed /etc/passwd by user1
accessed /var/www/html/index.php by www-data
```

```
sensitive = ["/etc/shadow", "/etc/passwd"]
with open("file_audit.log") as f:
    for line in f:
        for path in sensitive:
            if path in line:
                print("[ALERT] Sensitive file accessed:", line.strip())
```

---

6. Detecting Command Execution Patterns

Identify attacker tools or suspicious binaries in logs (e.g., `wget`, `curl`, `netcat`).

```
indicators = ["wget", "curl", "nc", "nmap", "python -m http.server", "scp", "sshpass"]
with open("bash_history.log") as f:
    for line in f:
```

```
        if any(cmd in line for cmd in indicators):
            print("[SUSPICIOUS CMD]", line.strip())
```

---

7. Correlating Event Sequences

Sometimes the **sequence of events** matters more than single lines.

Example logic:
If:

- There are failed login attempts from an IP, and then
- A successful login from the same IP

...it may indicate a successful brute-force.

```
import re
from collections import defaultdict

failures = defaultdict(int)
successes = {}

with open("auth.log") as f:
    for line in f:
        if "Failed password" in line:
            match = re.search(r"from (\d+\.\d+\.\d+\.\d+)", line)
            if match:
                ip = match.group(1)
                failures[ip] += 1
        elif "Accepted password" in line:
            match = re.search(r"from (\d+\.\d+\.\d+\.\d+)", line)
            if match:
```

```
                ip = match.group(1)
                successes[ip] = True

for ip in successes:
    if failures[ip] >= 5:
        print(f"[CORRELATED ALERT] Brute-force followed by success from {ip}")
```

---

8. Data Exfiltration Detection

Monitor unusually large downloads or transfers.

Assuming this CSV structure:

```
timestamp,user,file,bytes_transferred
2025-06-14T10:11:00Z,alice,secrets.zip,10500000
```

```
import csv

threshold = 10 * 1024 * 1024  # 10 MB

with open("downloads.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        if int(row["bytes_transferred"]) > threshold:
            print(f"[EXFIL ALERT] {row['user']} downloaded {row['file']} of size {row['bytes_transferred']}
bytes")
```

---

9. Aggregating Multiple Events per User

```python
from collections import defaultdict

user_actions = defaultdict(list)

with open("activity.log") as f:
    for line in f:
        if "accessed" in line:
            user = line.split("by")[-1].strip()
            user_actions[user].append(line.strip())

for user, actions in user_actions.items():
    if len(actions) > 10:
        print(f"[ALERT] Unusual activity by {user}: {len(actions)} actions")
```

---

10. Exporting Suspicious Activity

Export alerts to JSON for SIEM ingestion.

```python
import json

alerts = [
    {"type": "brute_force", "ip": "192.168.1.10", "attempts": 8},
    {"type": "sensitive_file", "user": "root", "file": "/etc/shadow"}
]

with open("alerts.json", "w") as f:
    json.dump(alerts, f, indent=2)
```

Conclusion

Suspicious behavior detection is essential to modern defensive security. It enables:

- Faster incident triage
- Proactive threat hunting
- Automated alerts in SIEMs and SOAR platforms

Combining techniques like frequency analysis, pattern matching, and event correlation with Python empowers analysts to build effective detection logic even without expensive commercial tools.

Practice Task

Create a Python script that:

1. Monitors an SSH log for failed and successful logins.
2. Detects brute-force followed by success.
3. Flags activity outside 8 AM – 6 PM.
4. Logs all suspicious activity to a JSON file.

3.3 - Event Correlation Across Multiple Sources

Objective

To understand and implement techniques for correlating security events from multiple log sources to detect complex attack patterns, reduce false positives, and gain contextual insight into incidents.

This topic builds upon log normalization and behavior detection by aggregating data from different systems (e.g., firewalls, endpoint logs, authentication systems, cloud services) and identifying meaningful relationships among them.

1. Introduction to Event Correlation

Event correlation is the process of linking events from different sources to build a broader understanding of what's happening in an environment. A single event might look benign, but when viewed in the context of other data sources, it may indicate malicious behavior.

Example:

- A firewall log shows a connection from an external IP.
- An endpoint log shows a process spawning PowerShell soon after.
- A DLP system detects a ZIP file being uploaded.

Individually, these events might be ignored. Together, they strongly indicate data exfiltration.

---

2. Key Concepts in Event Correlation

Concept	Description
Time correlation	Events occur close together (e.g., 10 seconds apart)
Entity correlation	Events involve the same user, host, IP address, or file
Sequence correlation	Events happen in a specific order (e.g., login → privilege escalation)
Threat context	Mapping events to known attack patterns (MITRE ATT&CK, IOC feeds)

---

3. Common Log Sources to Correlate

- **Authentication logs** (e.g., `/var/log/auth.log`)

- **Firewall logs** (e.g., iptables, pfSense)
- **EDR logs** (e.g., process creation, file access)
- **Web server logs** (e.g., Apache/Nginx access logs)
- **DNS queries**
- **Cloud provider logs** (e.g., AWS CloudTrail)
- **Email gateway logs**
- **SIEM feeds** (e.g., from ELK, Splunk)

---

4. Parsing and Normalizing Logs from Different Sources

To correlate logs, you must first bring them to a **standard format**, like:

```
{
    "timestamp": "2025-06-14T16:20:00Z",
    "source": "auth",
    "user": "alice",
    "event": "failed_login",
    "ip": "192.168.1.50"
}
```

Let's normalize a few sources.

Auth log example:

```
import re
from datetime import datetime

def parse_auth_line(line):
    match = re.search(r"(\w+\s+\d+)\s[\d:]+\s.*Failed password.*from (\d+\.\d+\.\d+\.\d+)", line)
```

```
    if match:
        dt = datetime.strptime(f"2025 {match.group(1)}", "%Y %b %d")
        return {
            "timestamp": dt.isoformat(),
            "source": "auth",
            "event": "failed_login",
            "ip": match.group(2)
        }
    return None
```

Web access log example:

```
192.168.1.100 - - [14/Jun/2025:16:20:30 +0000] "GET /admin HTTP/1.1" 200 305
```

```
def parse_web_line(line):
    match = re.search(r"(\d+\.\d+\.\d+\.\d+).*\[(.*?)\] \"GET (.*?)\"", line)
    if match:
        dt = datetime.strptime(match.group(2), "%d/%b/%Y:%H:%M:%S %z")
        return {
            "timestamp": dt.isoformat(),
            "source": "web",
            "event": "url_access",
            "ip": match.group(1),
            "url": match.group(3)
        }
    return None
```

---

5. Storing Normalized Logs

Save parsed logs into a common structure (e.g., list or database).

```
all_events = []

# Simulated reading
auth_lines = open("auth.log").readlines()
web_lines = open("access.log").readlines()

for line in auth_lines:
    event = parse_auth_line(line)
    if event: all_events.append(event)

for line in web_lines:
    event = parse_web_line(line)
    if event: all_events.append(event)
```

---

6. Basic Correlation Example: Same IP, Multiple Sources

```
from collections import defaultdict

ip_events = defaultdict(list)

for event in all_events:
    ip_events[event["ip"]].append(event)

# Alert on IPs seen in more than one log source
for ip, events in ip_events.items():
    sources = set(event["source"] for event in events)
    if len(sources) > 1:
        print(f"[CORRELATION ALERT] IP {ip} involved in multiple sources:")
```

```
        for e in events:
            print("  ", e)
```

This helps detect lateral movement or multi-vector attacks.

---

7. Time-Based Correlation

Let's identify sequences of events that happen close in time.

```
from datetime import datetime, timedelta

# Example: suspicious auth followed by /admin access within 1 minute
for ip, events in ip_events.items():
    events.sort(key=lambda x: x["timestamp"])
    for i in range(len(events) - 1):
        a, b = events[i], events[i + 1]
        if a["event"] == "failed_login" and b["event"] == "url_access" and "/admin" in b.get("url", ""):
            time_diff = datetime.fromisoformat(b["timestamp"]) - datetime.fromisoformat(a["timestamp"])
            if time_diff <= timedelta(minutes=1):
                print(f"[TIME CORRELATION] {ip} failed login then accessed admin in {time_diff}")
```

---

8. Cross-User Correlation

Track accounts that log in from unusual IPs or use patterns seen in other accounts.

```
user_ips = defaultdict(set)

for event in all_events:
```

```
    if "user" in event:
        user_ips[event["user"]].add(event["ip"])

for user, ips in user_ips.items():
    if len(ips) > 2:
        print(f"[MULTIPLE IP ALERT] User {user} used multiple IPs: {ips}")
```

This could indicate compromised credentials being used across locations.

---

9. Output to JSON/CSV for Dashboards

```
import json

with open("correlated_events.json", "w") as f:
    json.dump(all_events, f, indent=2)
```

---

10. Practical Use Cases

Use Case	Correlation Logic
Brute force detection	Auth logs + firewall logs from same IP
Lateral movement	Multiple hosts accessed by same user/IP
Data exfiltration	DLP logs + downloads from same user
APT tracking	Match against known IOC IPs/domains
Web attacks	URL patterns + login attempts + file downloads

Conclusion

Correlating events across sources is critical for:

- **Reducing alert fatigue**
- **Identifying multi-stage attacks**
- **Providing actionable context**
- **Triggering automated responses in SIEM/SOAR**

By writing Python scripts to parse, normalize, and correlate logs, you can simulate powerful detection logic found in enterprise-grade security platforms.

Exercise

Write a Python script that:

1. Parses logs from auth, web, and process audit.
2. Normalizes and stores events in a list.
3. Detects:
   - Failed logins followed by `/admin` access.
   - Same IP seen in multiple sources.
   - Users accessing from >2 IPs.
4. Outputs all correlation alerts to a JSON file.

3.4 - Exporting Data to Dashboards (CSV, Excel, JSON)

Objective

Understand how to programmatically export parsed and correlated security data into common formats—CSV, Excel, and JSON—to facilitate visualization, dashboard integration, and analyst workflows.

This is the final step in many detection and monitoring pipelines: presenting the data in a structured, readable, and actionable way. Analysts or automated systems can then consume the data for deeper review, alert triaging, reporting, or long-term storage.

---

1. Why Export Data?

Exporting normalized and enriched log data is useful for:

- Feeding external dashboards (Kibana, Grafana, Splunk, etc.)
- Building ad-hoc pivot tables and charts (Excel/LibreOffice)
- Automating threat intelligence ingestion (via JSON feeds)
- Archiving incident artifacts
- Generating reports for auditors, management, or compliance

---

2. Choosing the Right Format

Format	Best For	Characteristics
CSV	Simplicity, spreadsheets	Flat structure, widely supported
Excel (XLSX)	Dashboards, charts, formatting	Supports multiple sheets, colors, formulas
JSON	Hierarchical data, API integration	Nested structure, easy to parse programmatically

---

3. Exporting to CSV

CSV is a simple, human-readable format that's ideal for flat log structures.

Code Example: Exporting a List of Dicts

```
import csv

# Simulated normalized logs
events = [
    {"timestamp": "2025-06-14T16:22:00", "source": "auth", "event": "failed_login", "user": "alice", "ip":
"10.0.0.5"},
    {"timestamp": "2025-06-14T16:22:10", "source": "web", "event": "url_access", "user": "alice", "url":
"/admin", "ip": "10.0.0.5"}
]

with open("events_export.csv", "w", newline="") as csvfile:
    fieldnames = events[0].keys()
    writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

    writer.writeheader()  # Write column names
    for event in events:
        writer.writerow(event)
```

Notes:

- `DictWriter` handles writing dictionary keys as column headers.
- Ensure all dictionaries in the list have the same keys to avoid errors.

---

4. Exporting to Excel (XLSX)

The Excel format enables richer formatting, conditional highlights, charts, and multiple sheets. Python provides `openpyxl` and `xlsxwriter` for this.

Code Example: Exporting Logs to Excel

```
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill

wb = Workbook()
ws = wb.active
ws.title = "Log Events"

# Header row
headers = list(events[0].keys())
ws.append(headers)

# Bold + fill style
header_style = Font(bold=True)
for col in range(1, len(headers)+1):
    ws.cell(row=1, column=col).font = header_style
    ws.cell(row=1, column=col).fill = PatternFill(start_color="D9E1F2", fill_type="solid")

# Write rows
for event in events:
    ws.append([event.get(h, "") for h in headers])

wb.save("events_export.xlsx")
```

Notes:

- Use conditional formatting to highlight suspicious rows (e.g., failed logins, access to `/admin`)
- Each worksheet can represent a log source (e.g., Auth, Web, EDR)

5. Exporting to JSON

JSON is ideal for nested, structured data and interoperability between systems.

Code Example: Exporting to JSON File

```
import json

with open("events_export.json", "w") as f:
    json.dump(events, f, indent=2)
```

Notes:

- Use `indent=2` for human-readable formatting.
- JSON supports nested fields (like lists, dictionaries inside logs).

---

6. Advanced Export Use Case: Multi-Sheet Excel Dashboard

Suppose we have logs from different sources and want to split them into sheets.

```
from openpyxl import Workbook

sources = {
    "auth": [e for e in events if e["source"] == "auth"],
    "web": [e for e in events if e["source"] == "web"]
}

wb = Workbook()
for i, (source, rows) in enumerate(sources.items()):
```

```
    ws = wb.create_sheet(title=source) if i > 0 else wb.active
    ws.title = source
    headers = list(rows[0].keys())
    ws.append(headers)
    for row in rows:
        ws.append([row.get(h, "") for h in headers])

wb.save("multisource_logs.xlsx")
```

---

7. Integration with Dashboards and SIEM

Dashboards often ingest data via:

- **REST APIs**: Feed JSON or CSV data into tools like Splunk HEC, Elastic Beats, Graylog Input API
- **File Watchers**: Export files to watched directories
- **Database Sync**: Store to SQLite/Postgres and connect visualization tools (e.g., Metabase, Grafana)

---

8. Best Practices for Export Pipelines

Practice	Description
Sanitize outputs	Avoid newline injections, escape delimiters (esp. in CSV)
Timestamp formatting	Use ISO-8601 (YYYY-MM-DDTHH:MM)
Column consistency	Use the same order and naming across exports
Metadata tagging	Include source, sensor_id, or pipeline_id in the data

Practice	Description
Compression	Use .zip, .gz or .xz for large exports

---

9. Real-World Scenarios

Scenario	Export Target	Goal
Daily incident report	Excel file with charts	Analyst review & ticketing
API-based threat intel	JSON array per event	Ingest into SOAR/IR playbooks
Compliance reports	CSV with timestamped entries	Share with audit or compliance team
Dashboards	CSV or SQL backend	Render on Grafana, Kibana, or Power BI

---

10. Example: Combining Correlation and Export

```
# After correlation analysis
alerts = [
    {"ip": "10.0.0.5", "issue": "failed login + /admin access", "user": "alice", "time": "2025-06-14T16:22:10"},
    {"ip": "10.0.0.8", "issue": "multiple IPs for one user", "user": "bob", "time": "2025-06-14T18:12:50"}
]

# Export as CSV
with open("correlation_alerts.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=alerts[0].keys())
    writer.writeheader()
    writer.writerows(alerts)
```

Conclusion

Mastering data export techniques:

- Improves team visibility and response time
- Enables collaboration between SOC, IR, and compliance teams
- Supports integration with tools across the detection and response ecosystem

By using Python's `csv`, `json`, and `openpyxl` libraries, you can automate high-quality exports for every type of consumer: human analysts, APIs, dashboards, or regulators.

Exercise

Write a script that:

1. Parses logs and correlates them.
2. Outputs three versions:
   - CSV (flat view for Excel)
   - XLSX (separate sheets per log source)
   - JSON (nested, raw format)
3. Includes timestamps, source, user/IP, and detected behavior.

3.5 - Log Enrichment with External Intelligence

Objective

Enhance raw log data with contextual information from external sources such as threat intelligence feeds and open APIs. This process adds actionable insights and improves the accuracy and depth of incident detection.

Introduction

Log enrichment is the process of augmenting log entries with additional data points that are not present in the original source. This can include geolocation data, threat classifications, ASN ownership, service identification, and IP/domain reputational scores. When logs are enriched, SOC analysts and automated systems can make better decisions, correlate incidents more accurately, and prioritize threats effectively.

This topic focuses on using Python to enrich logs using publicly available APIs and external databases such as:

- VirusTotal (file, domain, IP reputation)
- AbuseIPDB (IP reputation and abuse categories)
- IPinfo / ip-api (geolocation, ASN)
- Shodan (open ports, device metadata)
- Custom threat feeds (STIX, TAXII, JSON, CSV)

---

1. Use Case: Enriching IPs from Firewall Logs

Imagine a log entry like:

```
192.168.1.10 -> 104.244.42.1:443 (allowed)
```

We want to enrich the IP `104.244.42.1` with ASN info, geolocation, and threat score.

---

2. Using ip-api.com for Geolocation Enrichment

```python
import requests

def enrich_with_ip_api(ip):
    url = f"http://ip-api.com/json/{ip}"
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        return {
            "country": data.get("country"),
            "region": data.get("regionName"),
            "city": data.get("city"),
            "org": data.get("org"),
            "asn": data.get("as")
        }
    else:
        return {"error": "Lookup failed"}

# Example
ip_data = enrich_with_ip_api("104.244.42.1")
print(ip_data)
```

 Commentary: This simple function queries `ip-api.com`, which does not require an API key. It retrieves key enrichment data like country, city, and ASN.

---

3. AbuseIPDB Lookup for Malicious IP Reputation

```python
import requests

API_KEY = "your_abuseipdb_api_key"
```

```python
def lookup_abuseipdb(ip):
    url = "https://api.abuseipdb.com/api/v2/check"
    headers = {
        "Accept": "application/json",
        "Key": API_KEY
    }
    params = {
        "ipAddress": ip,
        "maxAgeInDays": 90
    }
    response = requests.get(url, headers=headers, params=params)
    return response.json()

# Example
abuse_data = lookup_abuseipdb("104.244.42.1")
print(abuse_data['data']['abuseConfidenceScore'])
```

 Commentary: This score helps prioritize alerts. A score above 70 usually indicates confirmed abuse.

---

4. Shodan Integration for Surface Data

```python
import shodan

API_KEY = "your_shodan_key"
api = shodan.Shodan(API_KEY)

try:
    result = api.host("104.244.42.1")
    print("Open Ports:", result["ports"])
    print("OS Detected:", result.get("os"))
```

```
except shodan.APIError as e:
    print("Error:", e)
```

 Commentary: Shodan helps you understand the exposed services and fingerprinting of an IP, which can be essential during incident validation.

---

5. JSON Log Enrichment Workflow Example

```
import json

def enrich_log_entry(entry):
    ip = entry.get("dst_ip")
    geo = enrich_with_ip_api(ip)
    abuse = lookup_abuseipdb(ip)
    score = abuse['data']['abuseConfidenceScore']
    entry.update({
        "geo_country": geo.get("country"),
        "asn": geo.get("asn"),
        "abuse_score": score
    })
    return entry

# Example log
log = {"src_ip": "192.168.1.10", "dst_ip": "104.244.42.1", "action": "allowed"}
result = enrich_log_entry(log)
print(json.dumps(result, indent=2))
```

---

6. Security and Performance Considerations

- **Rate Limits:** Respect API rate limits to avoid bans
- **Caching:** Cache IP lookups locally to reduce API calls
- **Privacy:** Ensure sensitive data is handled per policy (GDPR, LGPD)
- **Error Handling:** Always add fallback if external services fail

---

7. Use Cases in Real Environments

- Enrich firewall logs to flag suspicious geolocations
- Enhance SIEM dashboards with IOC context
- Build threat scores for asset prioritization
- Flag rare countries or ASNs for alerting

---

8. Exercise

Create a script that:

1. Accepts a CSV file with a list of IPs
2. Enriches each IP using ip-api and AbuseIPDB
3. Writes output to a new CSV file with enrichment fields